

Introduction to Python

04 June 2026 17:04

Features of Python

- Python is a high level language. It is a free and open source language.
- It is an interpreted language, as Python programs are executed by an interpreter.
- Python programs are easy to understand as they have a clear defined syntax and relatively simple structures.
- Python is case sensitive. For example Number and number are not same in Python.
- Python is portable and platform independent means it can run on various operating systems and hardware platforms.
- Python has a rich library of predefined functions.
- Python is also helpful in web development. Many popular web services and applications are built using Python.
- Python uses indentation for blocks and nested blocks.

It was initially designed by Guido van Rossum in 1991 and developed by the Python Software Foundation. The language was conceived in the late 1980s, and its implementation began in December 1989 at Centrum Wiskunde & Informatica (CWI) in the Netherlands.

Working with Python

To write and run (execute) a program. We need to have a Python interpreter installed in our computer. Or we can use any online Python Interpreter.

```
Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>> 3 + 8
11
>>> print(4 + 5)
9
>>> print('Hello!')
Hello!
>>>
```

Handwritten annotations on the screenshot:

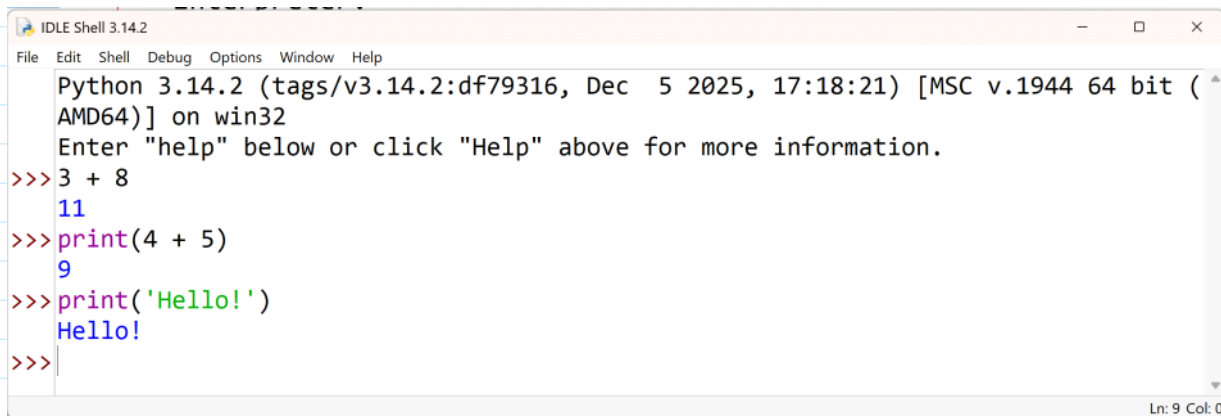
- REPL** (Read-Evaluate-Print-Loop) is written above the code.
- Read** points to the input prompt `>>>`.
- Evaluate** points to the code being executed.
- Print** points to the output displayed.
- Loop** points to the entire process.
- Accepts user input and parses it into Python expression.** points to the input prompt.
- Execute the parse code** points to the code being executed.
- Displays the result of execution.** points to the output displayed.
- Python interpreter or shell.** points to the window title.

Execution Mode:

- Interactive mode.
- Script mode.

- **Interactive Mode:** To work in the interactive mode, we can simply type a Python statement on the `>>>` Prompt directly. As soon as we press enter, the

interpreter executes the statement and displays the results.



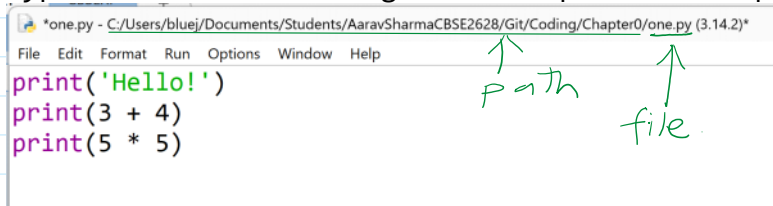
```
IDLE Shell 3.14.2
Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>> 3 + 8
11
>>> print(4 + 5)
9
>>> print('Hello!')
Hello!
>>>
```

Working in the interactive mode is convenient for testing a single line code for instant execution. But in the interactive mode, we cannot save the statements for future use and we have to retype the statements to run them again.

• Script Mode

In the script mode, we can write a Python program in a file, save it and then use the interpreter to execute it. Python scripts are saved as files where filename has extension ".py". By default, the Python scripts are saved in the Python installation folder. To execute a script we can either:

- Type the file name along with the path at the prompt.



```
*one.py - C:/Users/bluej/Documents/Students/AaravSharmaCBSE2628/Git/Coding/Chapter0/one.py (3.14.2)*
File Edit Format Run Options Window Help
print('Hello!')
print(3 + 4)
print(5 * 5)
```

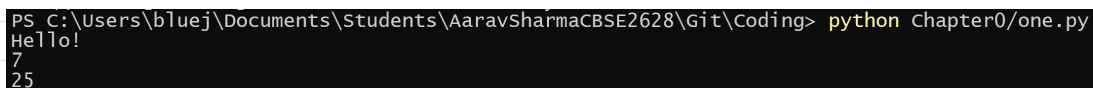
Handwritten annotations: 'Path' with an arrow pointing to the file path, and 'file' with an arrow pointing to the filename 'one.py'.

Source code

Output:

```
>>>
= RESTART: C:/Users/bluej/Documents/Students/AaravSharmaCBSE2628/Git/Coding/Chapter0/one.py
Hello!
7
25
>>>
```

Or



```
PS C:\Users\bluej\Documents\Students\AaravSharmaCBSE2628\Git\Coding> python Chapter0/one.py
Hello!
7
25
```

PYTHON Keywords

Keywords are reserved words. Each keyword has a specific meaning to Python interpreter and we can use a keyword in our program only for the purpose for which it has been defined. Tika as Python is case sensitive, keywords must be written exactly as given.

import keyword

```
print("Printing All Python keywords")
print("Python keywords are: ")
print(keyword.kwlist)
```

Output:

```
Printing All Python keywords
Python keywords are:
['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await', 'break',
'class', 'continue', 'def', 'del', 'elif', 'else', 'except', 'finally', 'for',
'from', 'global', 'if', 'import', 'in', 'is', 'lambda', 'nonlocal', 'not',
'or', 'pass', 'raise', 'return', 'try', 'while', 'with', 'yield']
```

```
import keyword
```

```
print("Printing All Python keywords")
print("Python keywords are: ")
keyWordlist = keyword.kwlist
print(keyWordlist)
print("Number of keyword:",len(keyWordlist))
```

= RESTART:

```
C:/Users/bluej/Documents/Students/AaravSharmaCBSE2628/Git/Coding/Chapter0/keywo
rd.py
```

```
Printing All Python keywords
Python keywords are:
['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await', 'break',
'class', 'continue', 'def', 'del', 'elif', 'else', 'except', 'finally', 'for',
'from', 'global', 'if', 'import', 'in', 'is', 'lambda', 'nonlocal', 'not',
'or', 'pass', 'raise', 'return', 'try', 'while', 'with', 'yield']
Number of keyword: 35
```

Identifiers or variables

In programming languages, identifiers are names used to identify a variable, function or other entities in Python program. The rules for naming an identifier in Python are as follows:

- The name should begin with an uppercase or a lowercase alphabet, or an underscore '_' sign. Thus an identifier cannot start with a digit.
- It can be of any length. (However, it is preferred to keep it short and meaningful.)
- It should not be a keyword or reserve word.
- We cannot use special symbols like !, @, #, \$, % etc. in identifiers.

Variables: A variable in a program is uniquely identified by a name (identifier). Variable in Python refers to an object - an item or element that is stored in the memory. Value of the variable can be string, numeric or any combination of alphanumeric characters. In Python we can use an assignment statement to create new variables and assign a specific values to them.

For example:

```
message = 'Keep Smiling'
print(message)
userNo = 101
print(f'User number is {userNo}')
```

Output:

```
Keep Smiling
User number is 101
```

For variable declaration is implicit in Python means variables are automatically declared and defined when they are assigned a value the first time. Variables must always be assigned values before they are used in expression as otherwise it will lead to an error in the program. So whenever a variable name occurs in an expression, the interpreter replaces it with the value of the particular variable.

← *Comment* ← Python interpreter will skip this line for interpretation.

```
# Write a Python program to find the area of a rectangle given that its
# length is 10 units and breadth is 20 units.
# main_code
length = int(input('Enter length: '))
breadth = int(input('Enter breadth: '))
area = length * breadth
print(f'area of rectangle: {area} sq.units')
```

Output:

```
Enter length: 23
Enter breadth: 17
area of rectangle: 391 sq.units
```

Everything is an object

Python treats every value or data item, whether numeric, string or other type. As an object in the sense that it can be assigned to some variable or can be passed to a function as an argument.

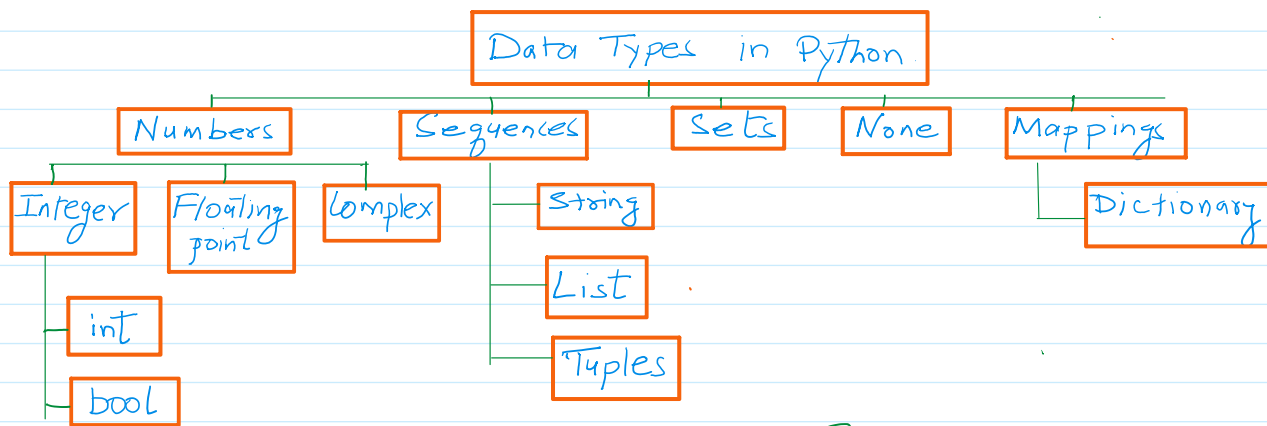
Every object in Python is assigned a unique identity (ID) which remains the same for the lifetime of that object. The function `id()` returns the identity of an object.

```
>>> name = 10
>>> type(name)
<class 'int'>
>>> id(name)
140711393785240
>>> name = 'Aarav'
>>> type(name)
<class 'str'>
>>> id(name)
2843222757744
>>> x = 10
>>> id(x)
140711393785240
>>> x = 11
>>> id(x)
140711393785272
```

For every distinct value a separate id is maintained by the Python automatically.

Data types

Every value belongs to a specific data type in Python. Data type identifies the type of data values a variable can hold and the operation that can be performed on that data. The following figure enlist the data type available in Python.



Different data types in Python.

$$\sqrt{-25} = \sqrt{(-1)(25)}$$

$\begin{matrix} 5 & \times & i \\ \hline 5 & i \end{matrix}$

Number

Number data type stores numerical values only. It further classified into three different types: bool, int, float and complex.

Numeric data types

Type/Class	Description	Examples
int	Integer numbers	-12, 0, -3, 15, 29, and many more
float	Real or floating point numbers	-2.05, 4.0, 14.23, many more
complex	complex number	<pre>>>> c = 5j >>> type(c) <class 'complex'> >>> d = 0j >>> type(d) <class 'complex'></pre>
bool	Boolean data type is a sub-type of integer. It can store two values either True or False.	<pre>>>> m = True >>> type(m) <class 'bool'> >>> n = False >>> type(n) <class 'bool'></pre>

Sequence

A Python sequence is an ordered collection of items, where each item is indexed by an integer. The three types of sequence data types available in Python are

strings, list and tuples.

A brief introduction to these data types is as follows:

String: String is a group of characters. These characters may be alphabets, digits, or special characters including spaces. String values are enclosed either in single quotation marks or in double quotation marks. The codes are not a part of strength. They are used to mark the beginning and end of the string for the interpreter. For example:

```
>>> name = "Aarav"
>>> name1 = 'Ajay'
>>> name
'Aarav'
>>> name1
'Ajay'
```

List: List is a sequence of items separated by commas and the items are enclosed in brackets [].

```
>>> li = []
>>> li1 = [5, 10.5, 34, -19, 'Hello', '20C', 200]
>>> li1
[5, 10.5, 34, -19, 'Hello', '20C', 200]
>>> li
[]
>>>
>>> li2 = [[1,2,3], [4,5,6], 100, 'Aarav']
>>> print(li2)
[[1, 2, 3], [4, 5, 6], 100, 'Aarav']
>>> li2[0]
[1, 2, 3]
>>> li2[0][0]
1
>>> li2[0][1]
2
```

Tuple

Tuple is a sequence of items separated by commas and items are enclosed in parentheses (). This is unlike list where values are enclosed in brackets []. Once created, we cannot change the tuple.

```
>>> tuple2 = (10, 20, 'Apple', [1,2,30], 'Hello', 3.5, -5j)
>>> print(tuple2)
(10, 20, 'Apple', [1, 2, 30], 'Hello', 3.5, (-0-5j))
>>> tuple2[2]
tuple2[2]
>>> tuple2[2]
'Apple'
>>> tuple2[3]
[1, 2, 30]
>>> tuple2[6]
(-0-5j)
>>> tuple2[3][2]
30
```

Set

Set is an unordered collection of items separated by commas, and the items are enclosed in curly brackets {}. A set is similar to list except that it cannot have duplicate entries. Once created, elements of a set cannot be changed.

For example:

```
>>> set1 = {23, 3.14, 'New Delhi', (12, 14, 16)}
>>> print(set1)
{'New Delhi', 3.14, (12, 14, 16), 23}
>>> set2 = {1,2,1,3}
>>> set2
{1, 2, 3}
```

None: None is a special data type with a single value. It is used to signify the absence of value in a situation. Python supports no special operations and it is neither False nor 0.

```
>>> myvar = None
>>> type(myvar)
<class 'NoneType'>
>>> myvar
>>> print(myvar)
None
```

Mapping: Dictionary in Python holds data items in key - value pairs. Items in dictionaries are enclosed in curly brackets {}. Dictionaries permit faster access to data. Every key is separated from its value using a colon (:) sign. The key: value pairs of a dictionary can be accessed using the key. The keys are usually strings and their values can be any data type. In order to access any values in the dictionary we have to specify its key in brackets [].

```
>>> dict1 = {'Fruit':'Mango', 'Climate':'Hot and humid', 'Price(Kg)':60}
>>> dict1
{'Fruit': 'Mango', 'Climate': 'Hot and humid', 'Price(Kg)': 60}
>>> dict1['Climate']
'Hot and humid'
>>> dict1['Price(Kg)']
60
```